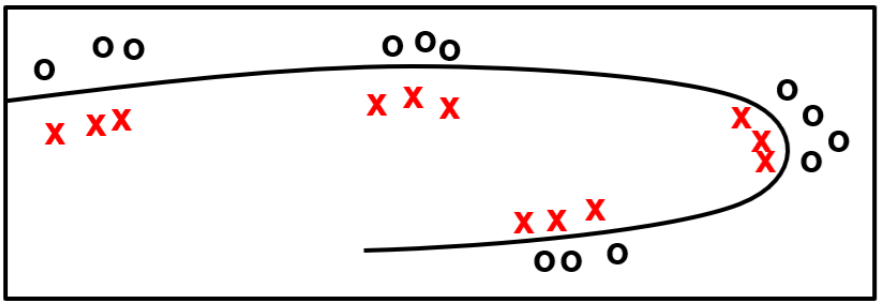
Modelos híbridos

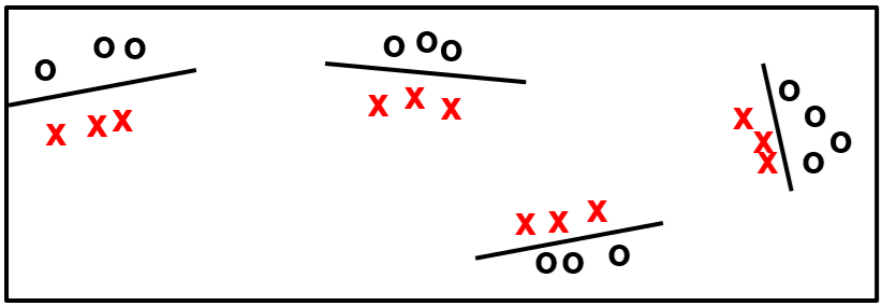
Modelos Avanzados de Aprendizaje Automático I

Los modelos híbridos de Aprendizaje Automático combinan diferentes técnicas y algoritmos de AA para aprovechar las fortalezas individuales de cada uno y mitigar sus debilidades. Los modelos híbridos buscan mejorar la precisión, la generalización y la capacidad de adaptación a distintos tipos de datos y problemas, y han destacado como una estrategia poderosa para mejorar el rendimiento y la robustez de los sistemas predictivos.

En el ejercicio anterior se implementaron modelos híbridos basados en árboles; en este ejercicio se implementarán modelos híbridos basados en kNN, para desarrollar modelos locales. La idea es que, para un problema concreto, el modelo global puede ser muy complejo; sin embargo, a nivel local los modelos pueden ser mucho más sencillos. Por ejemplo, como se muestra en teoría, para este *dataset* en un problema de clasificación la región de separación puede ser complicada:



Sin embargo, trabajando localmente, las regiones de separación a desarrollar podrían ser lineales:



Por lo tanto, se está suponiendo que los modelos locales a desarrollar son de baja complejidad. Esto, además, viene avalado por el hecho de que localmente el número de instancias suele ser mucho más bajo, por lo que un modelo complejo se sobreajustaría muy rápidamente. En sintonía con los ejercicios anteriores, los modelos a desarrollar en este ejercicio se referirán a problemas de regresión.

Los modelos híbridos a desarrollar en este ejercicio se basarán en kNN, con 3 grandes componentes:

- Un modelo global que se genera en tiempo de entrenamiento.
- kNN para encontrar los vecinos más cercanos de la instancia de la cual se quiere realizar una predicción.
- Un modelo local que se entrena a partir de esos vecinos más cercanos cuando se quiere hacer una predicción.

Para desarrollar modelos híbridos basados en kNN, se dispondrá de la siguiente estructura:

```
struct KNN
    dataset                 :: Batch
    k                       :: Int
    globalModel             :: Any
    localModelConfiguration :: Dict
    alpha                   :: AbstractFloat
end
```

La explicación de los campos es la siguiente:

- *dataset*, de tipo *Batch*, con el conjunto de entrenamiento. El algoritmo de kNN propiamente dicho no se entrena, sino que almacena el conjunto de entrenamiento para recuperar las instancias más cercanas en tiempo de predicción.
- *k*, de tipo *Int*, con el número de vecinos más cercanos a utilizar.
- *globalModel*, de tipo *Any*, con el modelo global entrenado. Si no se quiere utilizar modelo global, este campo será igual a *nothing*.
- *localModelConfiguration*, de tipo *Dict*, con la configuración del modelo local. Si no se quiere usar modelo local, este campo contendrá la configuración correspondiente a usar como modelo local una constante, es decir, en este caso, la predicción local será igual a la media de las salidas deseadas de los vecinos más cercanos.
- *alpha*, de tipo AbstractFloat, con el valor de α usado para combinar predicción global y predicción local. Si no hay modelo global (el campo *globalModel* es igual a *nothing*), este valor se ignora. En cambio, si hay modelo global, según este valor se tendrán dos modalidades de modelo híbrido:
    - Si *alpha* no contiene valor (es igual a *NaN*), entonces se hará la variante explicada en clase de teoría como “Predicción global + Corrección Local”, donde el modelo local intenta corregir el modelo global.
    - Si *alpha* contiene valor, se hará la variante vista en teoría “Entrenamiento global del modelo base” donde los resultados de los modelos local y global se combinan usando

el valor de α de la siguiente manera:

$$\hat{y} = \alpha \cdot \hat{y}_{local} + (1 - \alpha) \cdot \hat{y}_{global}$$

De esta manera, esta estructura tiene una gran flexibilidad al permitir usar tanto el algoritmo kNN clásico, como las variantes mostradas en esta descripción. Para implementar este método, será necesario, al igual que en los ejercicios anteriores, desarrollar las funciones de entrenamiento (*trainRegressionModel*) y predicción (*predict*). La descripción de las funciones a desarrollar es la siguiente:

- *trainRegressionModel*. Función habitual de entrenamiento de modelos, en este caso para kNN. Recibe como parámetros de entrada:
    - El primer argumento será de tipo *Val{:KNN}*, e identifica la función que se quiere ejecutar como esta, para entrenar un kNN.
    - *configuration*, de tipo *Dict*, con la configuración del kNN. Además de la clave "modelType", que deberá ser igual a *:KNN* para que se llame a esta función, puede incluir las siguientes claves, todas de tipo *String*:
        - "k", con un valor entero con el número de vecinos más cercanos. Si no está presente, se tomará un valor de k=5.
        - "globalModelConfiguration", con un valor de tipo *Dict* con la configuración del modelo global. Si no está presente, no se usará el modelo global.
        - "localModelConfiguration", con un valor de tipo *Dict* con la configuración del modelo global. Si no está presente, se tomará un *Dict* con la configuración correspondiente a un modelo constante.
        - "alpha", con un valor de tipo *Real*, con el valor de α, que se usará para combinar modelo local y global. Si no está presente, se tomará de α = NaN.
    - *dataset*, de tipo *Batch*, con el conjunto de datos que define el problema.

  Para crear la estructura *KNN*, esto se puede hacer llamando a *KNN* como si fuera una función, pasándole como parámetros los valores de los campos en el mismo orden en que han sido definidos, es decir, en este orden:
    - El *dataset* recibido como argumento.
    - El valor de *k* presente en *configuration*. Si esta clave no está presente como "k" (clave de tipo *String*), se tomará un valor por defecto de 5.
    - El modelo global entrenado. Por lo tanto, será necesario entrenarlo, llamando a la

función *trainRegressionModel* indicando, como primer argumento, la configuración del modelo global, y, como segundo argumento, el *dataset*. La configuración del modelo global estará en la clave "globalModelConfiguration" de *configuration*. Si no está presente, no se entrenará el modelo global, y, en lugar de ello, este campo deberá ser igual a *nothing*.

- La configuración del modelo local, a ser entrenado para cada predicción. Esta configuración estará en la clave "localModelConfiguration" de *configuration*. Si no está presente, se tomará el modelo básico usado en kNN estándar: hacer el promedio de las salidas deseadas. Esto se consigue pasando como configuración la del modelo constante, con *Dict("modelType" => :constant)*
- El valor de α, que se tomará del valor de la clave “alpha” de *configuration*. Si esta clave no está presente, se tomará *NaN*.

De esta manera, dado un dataset, se muestran a continuación distintos ejemplos de llamadas a esta función para crear modelos híbridos:

- Para utilizar el algoritmo clásico de kNN, sin modelo global ni local:

```
model = trainRegressionModel(
    Dict(
        "modelType" => :KNN,
        "k"         => 3
    ), dataset)
```

- Para utilizar como modelo global DoME, que se corrija localmente mediante la predicción local clásica de kNN (promedio de las salidas deseadas de los vecinos más cercanos):

```
model = trainRegressionModel(
    Dict(
        "modelType" => :KNN,
        "k"         => 6,
        "globalModelConfiguration" => Dict(
            "modelType"    => :DoME,
            "maximumNodes" => 40
        )
    ), dataset)
```

- Para no usar modelo global, y que el modelo local sea un SVR con *kernel* lineal (en

este caso, se devuelve el resultado del modelo local):

```
model = trainRegressionModel(
    Dict(
        "modelType" => :KNN,
        "k"         => 7,
        "localModelConfiguration" => Dict(
            "modelType" => :SVR,
            "epsilon"   => 0.1,
            "C"         => 0.01,
            "kernel"    => "linear"
        )
    ), dataset)
```

- Para usar como modelo global una constante, y que el modelo local sea lineal, combinándose la salida de ambos modelos con $\alpha = 0.2$:

```
model = trainRegressionModel(
    Dict(
        "modelType" => :KNN,
        "k"         => 5,
        "globalModelConfiguration" => Dict(
            "modelType" => :constant
        ),
        "localModelConfiguration" => Dict(
            "modelType" => :linear
        ),
        "alpha" => 0.2
    ), dataset)
```

- Para usar como modelo global uno lineal, y como local DoME, donde el local intenta corregir los errores del global:

```
model = trainRegressionModel(
    Dict(
        "modelType" => :KNN,
        "k"         => 4,
```

```
        "globalModelConfiguration" => Dict(
            "modelType" => :linear
        ),
        "localModelConfiguration" => Dict(
            "modelType"    => :DoME,
            "maximumNodes" => 50
        )
    ), dataset)
```

- Para mostrar la capacidad del sistema desarrollado en estos ejercicios, se muestra un ejemplo más complejo en el que se crea un árbol de modelos, donde cada modelo local, a su vez, es un KNN con modelos "global" y local:

```
model = trainRegressionModel(
    Dict(
        "modelType"               => :ModelTree,
        "max_depth"               => 4,
        "min_samples_leaf"        => 20,
        "localModelConfiguration" => Dict(
            "modelType" => :KNN,
            "k"         => 3,
            "globalModelConfiguration" => Dict(
                "modelType" => :linear
            ),
            "localModelConfiguration" => Dict(
                "modelType"    => :DoME,
                "maximumNodes" => 50
            )
        )
    ), dataset
)
```

- De igual manera, se podría crear un kNN donde el modelo global o local fuese un árbol de modelos, o incluso un kNN donde el modelo global o local fuese otro kNN, etc.

Para el desarrollo de esta función no se permite el uso de bucles.

- *predict*. La función estándar utilizada para hacer predicciones, que en este caso debe ser implementada para la estructura *KNN* desarrollada. Dado que en kNN la evaluación de un *dataset* no se puede realizar de forma matricial, puesto que cada instancia tendrá unos vecinos más cercanos distintos, es necesario sobrecargarla, implementando en primer lugar el método que recibe una única instancia, como un vector, y hace una única predicción. Esta recibe los siguientes argumentos:
    - *model*, de tipo *KNN*, con la estructura del kNN.
    - *inputs*, de tipo *AbstractArray{<:Real,1}*, con un vector con una instancia.

  Este método implementará la lógica de kNN como modelo híbrido descrita en este documento. Para realizar esta implementación, para una única instancia (un vector), hay que tener en cuenta estos aspectos:
    - El subconjunto del *dataset* con las instancias más cercanas se deberá tomar utilizando la función *nearestElements* desarrollada en un ejercicio previo.
    - Si hay modelo global, pero α no tiene valor, se desea que el modelo local corrija al global. En este caso, hay que modificar las salidas deseadas de este subconjunto que se va a usar para entrenar el modelo local para que estas tengan los residuos, es decir, la diferencia entre las salidas deseadas y las salidas del modelo global.
        - Para comprobar si existe modelo global, se puede utilizar la función *isnothing*. De forma similar, para comprobar si α tiene valor, se puede usar la función *isnan*.
    - Siempre habrá un modelo local. Según cómo está definida la función anterior, si no se especifica un modelo local, este será el modelo *:constant* definido en los ejercicios previos, es decir, tomar el valor promedio de la salida deseada de los vecinos más cercanos.
    - El modelo local se entrenará con la función *trainRegressionModel*, tras lo cual se hará una predicción en la instancia con la función *predict*. Para esto último, hay que convertirla de vector a matriz con una fila. Como resultado, devolverá un vector de un elemento, por lo que habrá que tomar ese elemento.
    - Si no hay modelo global, se devolverá ese elemento. En caso contrario, se aplicará el modelo global (función *predict*) sobre esa instancia (teniendo en cuenta lo anterior de la aplicación de *predict* sobre un vector, es decir, habrá que convertirlo a una matriz), y se devolverá la combinación de la predicción local y global, según el valor de α.

Esta función deberá devolver un valor real, no un vector ni una matriz. Para el desarrollo de esta función no se permite el uso de bucles.

- *predict*. Como segundo método a implementar en este ejercicio, esta función recibe, como segundo argumento, una matriz de entradas (de tipo *AbstractArray{<:Real,2}*), en lugar de un vector con una única instancia. Esta función se implementará fácilmente mediante una *comprehension* que itere sobre las filas de la matriz de entradas utilizando la función *eachrow*, y haciendo llamadas a la función anterior. Para el desarrollo de esta función no se permite el uso de bucles.

Firmas de las funciones:

A continuación se muestran las firmas de las funciones a realizar en los ejercicios propuestos. Tened en cuenta que, dependiendo de cómo se defina la función, esta puede contener o no la palabra reservada *function* al principio:

```
function trainRegressionModel(::Val{:KNN}, configuration::Dict, dataset::Batch)
function predict(model::KNN, instance::AbstractArray{<:Real,1})
function predict(model::KNN, inputs::AbstractArray{<:Real,2})
```